

How to Create Custom Exceptions in C# with examples

In this article, I am going to discuss **How to Create Custom Exceptions in C#** with Examples. Please read our previous article where we discussed how to implement **Finally Block in C#** with Examples. As part of this article, we are going to discuss the following pointers in detail.

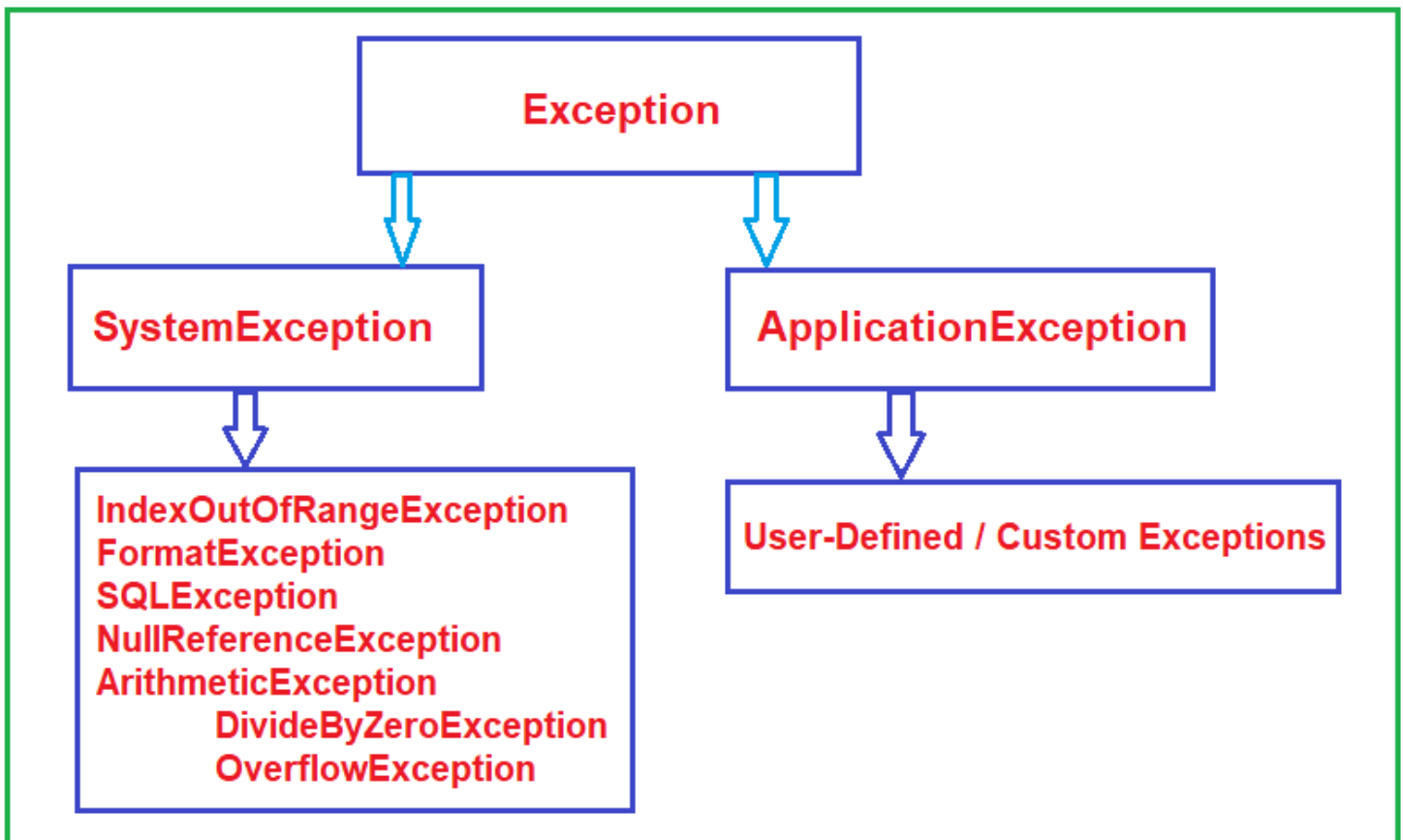
1. **What are System Exceptions and Application Exceptions in C#?**
2. **How to create a Custom Exception in C#?**
3. **Why do we need to create Custom Exceptions in C#?**
4. **Examples to Understand Custom Exceptions in C#.**

Types of Exceptions in C#:

Before understanding how to create Custom Exceptions or used Defined Exceptions in C#, let us first understand what are the different types of Exceptions available. In C#, the exceptions are divided into two types. They are as follows:

1. **System Exception:** These exceptions are caused by the CLR.
2. **Application Exception:** These exceptions are caused by the programmer.

For a better understanding, please have a look at the below image. The Exception is the parent class of all Exception classes. From the Exception class, SystemException and ApplicationException classes are defined.



As you can see, for both `SystemException` and `ApplicationException`, the parent is the `Exception` class only. By default, all the System Exception classes are inherited from the `SystemException` class which is inherited from the `Exception` class and if we are creating any Application Exception i.e. Custom Exception or user-defined exception, then that class should and must be inherited from the `Exception` class, even we can also create Custom Exception classes inherit from `ApplicationException` class. Previously we used to create `CustomException` classes inheriting from the `ApplicationException` class, but currently, Microsoft recommended to use the `Exception` class.

What are System Exceptions in C#?

An exception that is raised implicitly under a program at runtime by the Exception Manager (Component of CLR) because of some logical mistakes (some predefined conditions) is known as System Exception. For example, if you are dividing an integer number by zero, then one system exception is raised called `DivideByZero`. Similarly, if you are taking an alphanumeric string value from the user and trying to store that value in an integer variable, then one system exception is raised called `FormatException`. So, in C#, there are lots of System Exception classes are available. Some of them are as follows:

1. **`DivideByZeroException`**
2. **`IndexOutOfRangeException`**
3. **`FormatException`**
4. **`SQLException`**
5. **`NullReferenceException`, Etc.**

What are Application Exceptions in C#?

An exception that is raised explicitly under a program based on our own condition (i.e. user-defined condition) is known as an application exception. As a programmer, we can raise application exceptions at any given point in time. For example, our requirement is that while performing the division operation, we need to check that if the second number is an odd number, then we need to throw an exception. This cannot be handled automatically by the CLR. Then as a user, we need to create our Custom Exception and we need to create an instance of our Custom Exception class and we need to throw that Custom Exception instance using the `throw` keyword explicitly based on our business requirement.

To raise an Application Exception in C#, we need to adopt the following process. First, we need to create a custom Exception class by inheriting it from the Parent Exception class and then we need to create an instance of the Custom Exception class and then we need to throw that instance.

Step1: Create one Custom Exception Class

```
public class OddNumberException : Exception  
{
```

Step2: Create an instance of the Custom Exception Class

```
OddNumberException ONE = new OddNumberException();
```

Step3: Throw the Custom Exception Instance

```
throw ONE;  
throw new OddNumberException();
```

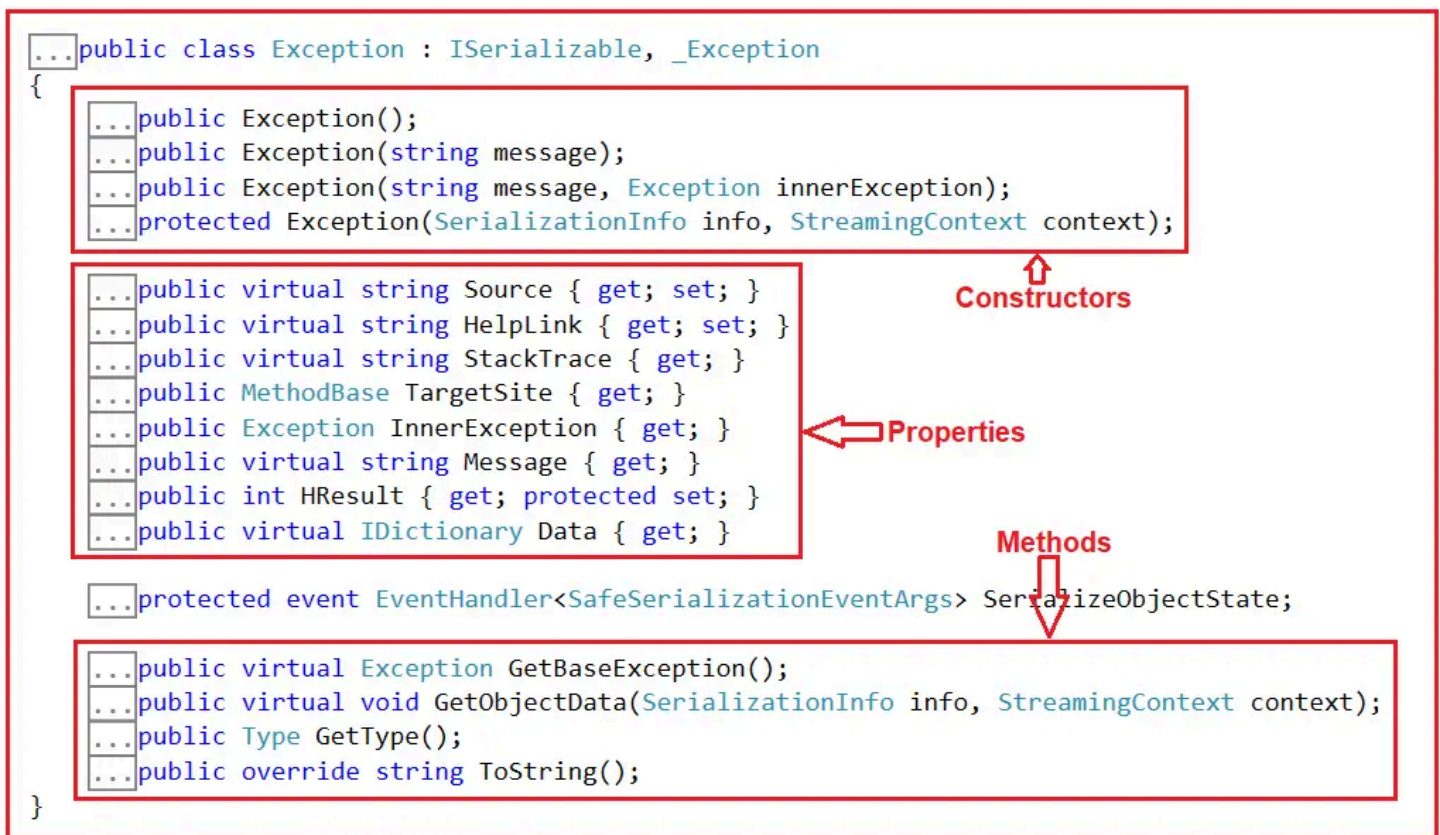
While creating and throwing an object of the Exception class manually, we should not use system exception classes like DivideByZeroException, FormatException, SQLException, etc. instead we should create our own Exception class and create and throw an instance of our own Exception class.

Why do we need to create Custom Exceptions in C#?

If none of the already existing .NET exception classes serve our purpose then we need to go for custom exceptions in C#.

How to Create Own Custom Exception Class in C#?

Before creating the Custom Exception class, we need to see the class definition of the Exception class as our Custom Exception class is going to be inherited from the parent Exception class. If you go to the definition of Exception class, then you will see the following.



As you can see, the `Exception` class has some constructors, some virtual and non-virtual properties, and some virtual and non-virtual methods. The virtual members you can override in the child of this `Exception` class and you can directly consume the non-virtual members using the child class instance.

Now, to create a Custom Exception class in C#, we need to follow the below steps.

1. **Step1:** Define a new class inheriting from the predefined `Exception` class so that the new class also acts as an `Exception` class.
2. **Step2:** Then as per your requirement, override the virtual members that are defined inside the `Exception` class like `Message`, `Source`, `StackTrace`, etc with the required error message.

Let us understand how to create a custom exception in C# with an example. Create a class file with the name `OddNumberException.cs` and then copy and paste the following code into it. Here, you can see that the `OddNumberException` class is inherited from the built-in `Exception` class and here we are re-implementing two virtual properties i.e. `Message` and `HelpLink`. Now, we can create an instance of `OddNumberException` class and if we invoke `Message` and `HelpLink` properties, then these two properties are going to be executed from this class only. But if you invoke the `Source` and `StackTrace` properties, then those properties are going to be executed from the `Exception` class only as we have not re-implemented these properties. This is the concept of **Method Overriding in C#**.

```
using System;
namespace ExceptionHandlingDemo
{
    //Creating our own Exception Class by inheriting Exception class
    public class OddNumberException : Exception
    {
        //Overriding the Message property
        public override string Message
```

```

    {
        get
        {
            return "Divisor Cannot be Odd Number";
        }
    }
    //Overriding the HelpLink Property
    public override string HelpLink
    {
        get
        {
            return "Get More Information from here:
https://dotnettutorials.net/lesson/create-custom-exception-csharp/";
        }
    }
}

```

Now, as per our business logic, we can explicitly create an instance of the OddNumberException class and we can explicitly throw that instance from our application code. For a better understanding, please have a look at the following code. Here, inside the Main method, we are taking two numbers from the user and then checking if the second number is odd or not. If the second number i.e. divisor is odd, then we are creating an instance of the OddNumberException class and throwing that instance. And in the Catch block, we are handling that exception and we simply printing Message, StackTrace, Source, and HelpLink properties. Here, if OddNumberException occurred, then Message and HelpLink properties are going to execute from OddNumberException class and Source and StackTrace properties are going to be executed from the pre-defined parent Exception class.

```

using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number:");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number:");
                Number2 = int.Parse(Console.ReadLine());
                if (Number2 % 2 > 0)
                {
                    //OddNumberException ONE = new OddNumberException();
                    //throw ONE;
                    throw new OddNumberException();
                }
                Result = Number1 / Number2;
                Console.WriteLine(Result);
            }
        }
    }
}

```

```

        catch (OddNumberException one)
        {
            Console.WriteLine($"Message: {one.Message}");
            Console.WriteLine($"HelpLink: {one.HelpLink}");
            Console.WriteLine($"Source: {one.Source}");
            Console.WriteLine($"StackTrace: {one.StackTrace}");
        }
        Console.WriteLine("End of the Program");
        Console.ReadKey();
    }
}
}

```

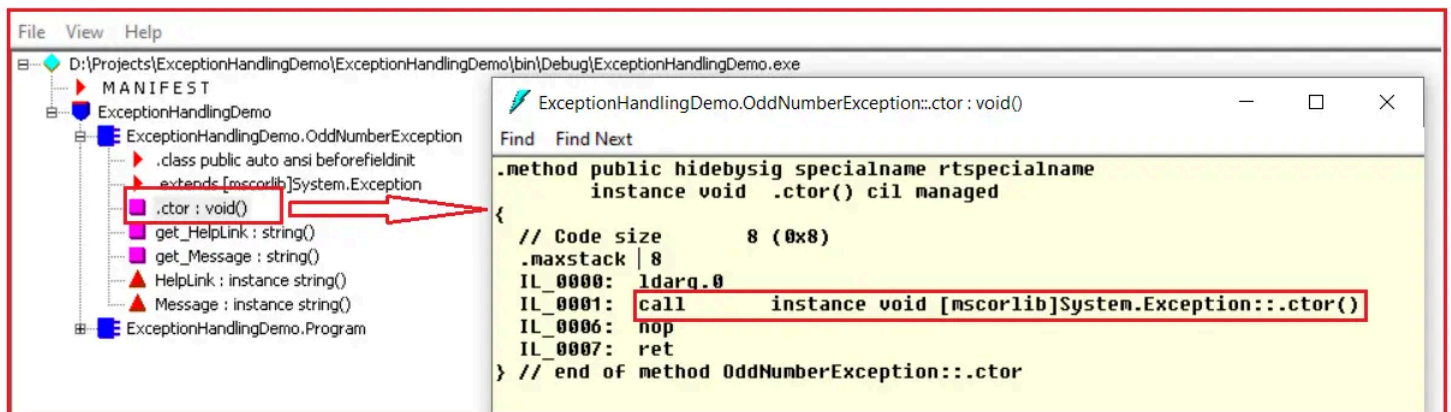
Output:

```

10
Enter Second Number:
3
Message: Divisor Cannot be Odd Number
HelpLink: Get More Information from here: https://dotnettutorials.net/lesson/create-custom-exception-c-sharp/
Source: ExceptionHandlingDemo
StackTrace:   at ExceptionHandlingDemo.Program.Main(String[] args) in D:\Projects\ExceptionHandlingDemo\ExceptionHandlingDemo\Program.cs:line 21
End of the Program

```

This fine. But if you see the Exception class, then you will see that the Exception class is having three public constructors. And in our example, we have used the overloaded constructor which does not take any parameter i.e. the default constructor. You might have one question. We have not used any constructor in our Custom Exception class, then how you can say, we are using the Exception class parameterless constructor? Yes, we are using it. This is implicit. And we have discussed this in our Constructor article. Whenever we are making the inheritance relationship the parent class constructor should be accessible to the child class otherwise inheritance is not possible. In our example, we have not defined any constructor explicitly, so the compiler will provide one constructor and that constructor implicitly calls the Exception class parameterless constructor. To prove this, open the IL Code of our Custom Exception class using the ILDASM tool and you will see the following.



You can see in the above IL code, our compiler provided a default constructor calling the Exception class parameterless constructor.

According to [Microsoft](#), When creating your own exceptions, end the class name of the user-defined exception with the word “Exception”, and implement the three common constructors. To do so, let us modify the OddNumberException class as follows to include three constructors which call the Exception class's respective constructors. Here, I am not re-implementing the Message virtual property and the reason is, by using the constructor only, I am going to pass the error message while creating the instance.

```
using System;
namespace ExceptionHandlingDemo
{
    //Creating our own Exception Class by inheriting Exception class
    public class OddNumberException : Exception
    {
        public OddNumberException()
        {
        }
        public OddNumberException(string message)
            : base(message)
        {
        }
        public OddNumberException(string message, Exception inner)
            : base(message, inner)
        {
        }

        //Overriding the HelpLink Property
        public override string HelpLink
        {
            get
            {
                return "Get More Information from here:
https://dotnettutorials.net/lesson/create-custom-exception-csharp/";
            }
        }
    }
}
```

In our previous example, we used the default constructor while creating an instance of OddNumberException class, now you can use the other overloaded version of the constructor which takes a message as a parameter as shown in the below code.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number:");
            }
        }
    }
}
```

```

        Number1 = int.Parse(Console.ReadLine());
        Console.WriteLine("Enter Second Number:");
        Number2 = int.Parse(Console.ReadLine());
        if (Number2 % 2 > 0)
        {
            //OddNumberException ONE = new OddNumberException();
            //throw ONE;
            throw new OddNumberException("Odd Number Exception Occured Inside the Main Method of Program Class");
        }
        Result = Number1 / Number2;
        Console.WriteLine(Result);
    }
    catch (OddNumberException one)
    {
        Console.WriteLine($"Message: {one.Message}");
        Console.WriteLine($"HelpLink: {one.HelpLink}");
        Console.WriteLine($"Source: {one.Source}");
        Console.WriteLine($"StackTrace: {one.StackTrace}");
    }
    Console.WriteLine("End of the Program");
    Console.ReadKey();
}
}
}

```

Now, run the application and pass the second number as an odd number and you will get the following output. Here, please observe the error message. Whatever message we are passing to the constructor that we are getting as part of the Message property.

```

100
Enter Second Number:
5
Message: Odd Number Exception Occured Inside the Main Method of Program Class
HelpLink: Get More Information from here: https://dotnettutorials.net/lesson/create-custom-exception-csharp/
Source: ExceptionHandlingDemo
StackTrace: at ExceptionHandlingDemo.Program.Main(String[] args) in D:\Projects\ExceptionHandlingDemo\ExceptionHandlingDemo\Program.cs:line 21
End of the Program

```

Can we create a Custom Exception class by Inheriting from ApplicationException class?

Yes. This is also possible. Even though Microsoft does not recommend to use to ApplicationException class to create a Custom Exception class, but still, we can create it. This is because ApplicationException class is inherited from the Exception. And if you go to the definition of ApplicationException class, then you will also see that this class is having those constructors as Exception class as shown in the below image.

```

...public class ApplicationException : Exception
{
    ...public ApplicationException();
    ...public ApplicationException(string message);
    ...public ApplicationException(string message, Exception innerException);
    ...protected ApplicationException(SerializationInfo info, StreamingContext context);
}

```

As this ApplicationException class is inherited from the Exception class, all the properties and methods available in Exception are also going to be available in the Custom Exception classes. Let us prove this. Simply modify the Custom class to be inherited from the ApplicationException instead of the Exception class as shown in the below code.

```

using System;
namespace ExceptionHandlingDemo
{
    //Creating our own Exception Class by inheriting Exception class
    public class OddNumberException : ApplicationException
    {
        public OddNumberException()
        {
        }
        public OddNumberException(string message)
            : base(message)
        {
        }
        public OddNumberException(string message, Exception inner)
            : base(message, inner)
        {
        }
        //Overriding the HelpLink Property
        public override string HelpLink
        {
            get
            {
                return "Get More Information from here:
https://dotnettutorials.net/lesson/create-custom-exception-csharp/";
            }
        }
    }
}

```

Now, with the above changes in place, run the application and you will also get the same output. So, these are the two ways to create Custom Exception Classes in C#.

In the next article, I am going to discuss the **Inner Exception in C#** with Examples. Here, in this article, I try to explain how to Create **Custom Exceptions in C#** with Examples. I hope you enjoy this How to Create Custom Exceptions in C# with Examples article.